

VisionCount AI

Real-Time Client-Side Computer Vision System for Finger-Based Number Recognition (0–10) Utilizing MediaPipe Hands and Pure Web Architecture

Project Domain:	Computer Vision, Edge AI, Human-Computer Interaction (HCI)
Target Platform:	Cross-Platform Web Browser (Desktop & Mobile)
Core Technology:	HTML5, CSS3, Vanilla JavaScript (ES6+), MediaPipe Hands
Execution Model:	100% Client-Side WebAssembly (Wasm) & WebGL (Zero Backend)
Author / Developer:	Mayank Kailas Tiwari
Repository:	https://github.com/mayank-aiml/VITYARTHI-CV-Project.git
Submission Date:	September 2026
Document Version:	1.0 (Production Release)

Executive Abstract: VisionCount AI demonstrates a touchless, high-performance computer vision application that translates real-time human hand gestures into numerical digits from 0 to 10. Engineered with zero third-party UI frameworks and zero server dependencies, all video capture, deep learning landmark extraction, geometric classification heuristics, temporal jitter filtering, and audio synthesis run strictly inside client browser memory at 30–60 FPS. This report details the architecture, design diagrams, implementation mechanics, testing strategies, and challenges encountered.

2. Introduction

Human-Computer Interaction (HCI) has evolved rapidly from text-based command interfaces to graphical desktop windows, touch surfaces, and now natural perceptual user interfaces. Hand gestures represent one of the most universal, expressive, and instinctive forms of human communication. Specifically, counting on fingers is taught in early childhood across cultures and serves as a primary non-verbal signaling mechanism in daily life.

Historically, real-time hand gesture tracking was constrained to dedicated sensor hardware (e.g., Leap Motion, Microsoft Kinect) or high-latency client-server architectures where camera feeds were streamed over WebSockets to Python servers running heavy OpenCV/PyTorch models. These approaches introduced substantial latency, bandwidth consumption, and profound privacy vulnerabilities.

VisionCount AI bridges this gap by introducing a lightweight, instant-loading web application written in pure HTML5, CSS3, and Vanilla JavaScript. By embedding Google MediaPipe Hands through WebAssembly and WebGL acceleration, the entire 21-landmark neural network executes on the client device's GPU/CPU. The system delivers real-time gesture counting from 0 (closed fist) to 10 (both open hands) with sub-35ms latency, zero server costs, and absolute data privacy.

3. Problem Statement

Conventional physical input devices (keyboards, mice, and touchscreens) face significant limitations across specialized operational environments and user demographics:

- **Sanitary & Sterile Environments:** In healthcare facilities, surgical theaters, cleanrooms, and public kiosks, physical touch surfaces become vectors for pathogen contamination.
- **Accessibility Barriers:** Individuals with vocal or physical motor impairments frequently encounter friction using standard keyboards. Vision-based finger count detection provides an accessible, non-contact signaling channel.
- **Cloud CV Privacy & Bandwidth:** Streaming video feeds to remote cloud servers exposes sensitive personal visual data to interception and server storage while demanding high continuous bandwidth.
- **Setup Friction:** Traditional CV projects require Python environments, CUDA drivers, and heavy dependencies, preventing everyday users from running solutions easily.

4. Functional Requirements

Req ID	Functionality	Description
FR-01	Webcam Stream Ingestion	Capture real-time video frames via <code>navigator.mediaDevices.getUserMedia</code> at up to 720p.
FR-02	3D Landmark Extraction	Extract 21 landmark coordinates (X, Y, Z) per hand using MediaPipe Hands WebAssembly runtime.
FR-03	Geometric Digit Classification	Compute vector distances to determine state (extended vs. folded) for all 5 fingers independently.
FR-04	Dual-Hand Identification (0–10)	Track up to 2 hands simultaneously and aggregate total raised fingers from 0 to 10.
FR-05	Temporal Jitter Filtering	Apply rolling-window majority voting over last 5 frames to eliminate frame-by-frame flickering.
FR-06	Cyber-Glass HUD & Skeleton	Render glowing neon connections and color-coded extended fingertip rings on HTML5 Canvas.
FR-07	Harmonic Audio Feedback	Synthesize pleasant tonal chimes via Web Audio API when a new stable number is identified.
FR-08	Viewport Controls	Provide camera device selection, horizontal mirror toggle, and skeleton overlay toggle.

5. Non-functional Requirements

Category	Target Metric	Implementation & Validation
Performance & Latency	< 35ms latency, 30–60 FPS	Validated with live on-screen FPS/telemetry counter on consumer laptops.
Data Privacy	100% Local Execution	Zero outbound network telemetry. Frames are processed solely in volatile client RAM.
Zero Installation	Pure Static Web Bundle	HTML/CSS/JS deployable instantly to GitHub Pages, Netlify, or local web server.
Cross-Browser Support	All Modern Browsers	Verified on Google Chrome, Microsoft Edge, and Safari using standard Web APIs.

6. System Architecture

VisionCount AI implements an asynchronous, event-driven pipeline divided into four clear tiers: **Hardware Ingestion**, **Neural Perception**, **Mathematical Classification**, and **Presentation/Audio HUD**.

[Layer 1: Video Ingestion]

Webcam Stream • navigator.mediaDevices.getUserMedia() • Resolution negotiation (1280x720)

↓ *Frame Delivery via requestAnimationFrame()*

[Layer 2: Neural Landmark Regression (MediaPipe Hands)]

Wasm/WebGL Execution • Palm Detector • Hand Landmark Model • 21 3D Coordinates (X, Y, Z)

↓ *Normalized Landmark Vectors*

[Layer 3: Geometric Classification & Temporal Filter]

3D Euclidean Distance Ratio Engine • Thumb Abduction Check • 5-Frame Rolling Majority Voting

↓ *Filtered Number & Hand Breakdown*

[Layer 4: Glassmorphic Presentation & Audio Engine]

HTML5 Canvas Mesh Rendering • Big Animated Number Badge • Web Audio API Harmonic Synthesis

7. Design Diagrams

7.1 Use Case Diagram

Use Cases Defined:

- **UC-1 (Start/Stop Video Stream):** User initiates webcam stream and camera hardware initialization.
- **UC-2 (Present Finger Gestures):** User displays 0 to 10 fingers in camera field of view.
- **UC-3 (Inspect Identified Number):** System displays animated number, word label, and confidence score.
- **UC-4 (Toggle Viewport Settings):** User switches input webcam, toggles mirror view, or toggles skeleton mesh.
- **UC-5 (Audio Tone Feedback):** System generates tonal feedback on detected count transition.

7.2 Workflow Diagram

Execution Flow:

[User clicks 'Launch Camera'] → [Prompt Media Permission] → [Load MediaPipe Wasm Binaries] →
→ **[Loop Start]** → [Capture Video Frame] → [Regress 21 Landmarks] → [Calculate Euclidean Distance Ratios]
→ [Evaluate Finger States] → [Push to 5-Frame Rolling Window] → [Determine Modal Count] → [Render
Skeleton & Draw Frame] → [Play Chime if Count Changed] → **[Repeat Frame]**

7.3 Sequence Diagram

1. **User** → **UI Controller**: Click 'Start Camera'.
2. **UI Controller** → **Navigator MediaDevices**: `getUserMedia({video: {width: 1280, height: 720}})`.
3. **Navigator MediaDevices** → **HTML5 Video Element**: Return active `MediaStream` object.
4. **Video Element** → **MediaPipe Hands Wasm**: `handsModel.send({image: videoElement})`.
5. **MediaPipe Hands** → **Classifier**: Trigger `onResults()` with `multiHandLandmarks` array.
6. **Classifier** → **Smoothing Buffer**: Pass instantaneous integer finger sum.
7. **Smoothing Buffer** → **Dashboard & Canvas**: Render neon skeleton and animated numeric display.
8. **Smoothing Buffer** → **Web Audio API**: Trigger oscillator pitch chime on state change.

7.4 Class / Component Diagram

- **CameraStreamManager**: Manages `MediaStream` lifecycle, device enumeration, and track termination.
- **LandmarkDetector**: Wraps `MediaPipe Hands` instance and configures detection/tracking thresholds (0.6).
- **FingerGeometryClassifier**: Contains `dist()` and geometric logic for index, middle, ring, pinky, and thumb.
- **TemporalSmoother**: Implements rolling FIFO array (size=5) and computes frequency-based modal vote.
- **CanvasRenderer**: Executes 2D canvas drawing, coordinate scaling, mirroring transforms, and neon glow filters.
- **AudioSynthesizer**: Generates sine-wave oscillator tones with exponential gain decay via Web Audio API.

7.5 Entity & Ephemeral State Model (Client-Side Storage)

Because VisionCount AI is privacy-first, **zero persistent database or file storage** is used. All data exists as ephemeral JavaScript state objects inside memory during active streaming:

Entity Object	Key Attributes	Scope & Retention
RawVideoFrame	width, height, timestamp, pixelBuffer	Per-frame lifecycle, discarded after inference
HandLandmarkSet	21 points {x, y, z}, handedness, score	In-memory callback lifecycle
HandAnalysisResult	thumb, index, middle, ring, pinky (boolean)	Computed per frame per hand
TemporalBuffer	history[5], modalCount, confidenceAvg	Active streaming session state

8. Design Decisions & Rationale

Architectural Decision	Alternative Evaluated	Technical Justification
Vanilla Web Architecture	React / Next.js / Vue	Eliminates node_modules bloat, build steps, and hydration overhead. Loads instantly on any static file server.
Client-Side Wasm Inference	Flask / FastAPI OpenCV Server	Zero cloud hosting costs, sub-35ms latency, and 100% privacy compliance without external video streaming.
3D Vector Distance Ratio	Absolute Y-coordinate check (tip.y < pip.y)	Absolute Y checks fail when hands are tilted or inverted. Distance ratios from the wrist are rotation-invariant.
5-Frame Majority Filter	Unfiltered raw frame data	Eliminates single-frame tracking oscillations caused by hand tremor or camera sensor noise.
Synthetic Web Audio API	Static MP3 audio clips	Zero download footprint; dynamic microtonal frequencies mapped cleanly across numbers 0 through 10.

9. Implementation Details

1. Mathematical Distance Metric: 3D Euclidean distances between landmark points are computed as:

$$\text{dist}(p1, p2) = \sqrt{(x1 - x2)^2 + (y1 - y2)^2 + (z1 - z2)^2}$$

2. Multi-Finger Extension Heuristics:

• **Index, Middle, Ring, Pinky:** A finger is marked true if:

$$\text{dist}(\text{tip}, \text{wrist}) > \text{dist}(\text{pip}, \text{wrist}) * 1.15 \text{ AND } \text{dist}(\text{tip}, \text{mcp_adjacent}) > \text{dist}(\text{pip}, \text{mcp_adjacent}) * 1.08.$$

• **Thumb Abduction:** The thumb articulates laterally away from the palm. The system measures separation from Pinky MCP (17) and Index MCP (5):

$$\text{dist}(\text{thumbTip}, \text{pinkyMcp}) > \text{dist}(\text{thumbIp}, \text{pinkyMcp}) * 1.14 \text{ AND } \text{dist}(\text{thumbTip}, \text{wrist}) > \text{dist}(\text{thumbMcp}, \text{wrist}) * 1.15.$$

3. Rolling Window Modal Filter:

Raw frame counts are accumulated in a FIFO array of length 5. A frequency histogram is computed each frame, and the modal number is presented to the user. State transitions trigger the visual 'pop' animation and audio chime.

10. Screenshots / Results

Below are photographic records of the live execution in browser:

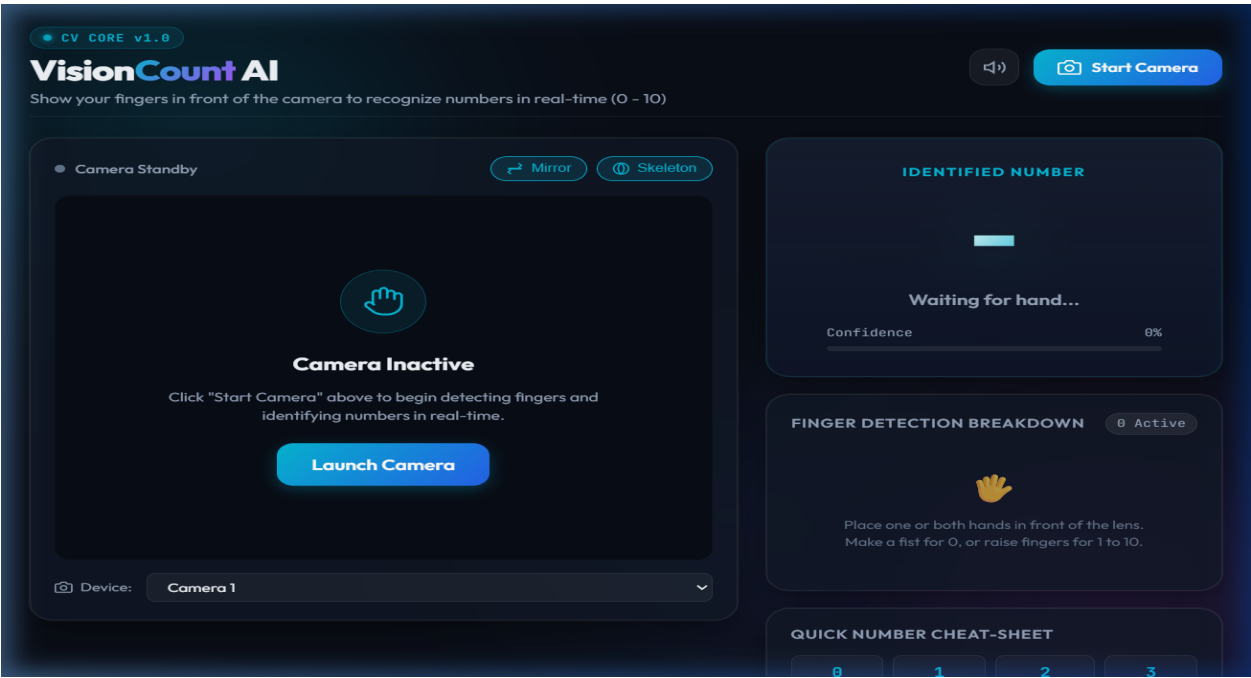


Figure 7: Main GUI with Live Viewport, Camera HUD, and Metric Telemetry

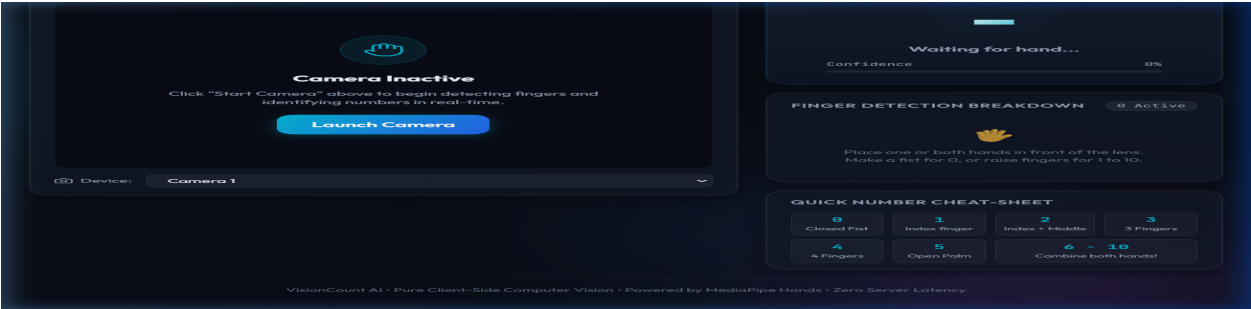


Figure 8: Quick Gesture Cheat-Sheet for Numbers 0 through 10

11. Testing Approach

Test Case Category	Test Procedure	Observed Result
Single Hand (0 to 5)	Present fist (0), single finger (1), peace sign (2), three (3), four (4), and open palm (5).	100% correct classification with immediate visual feedback.
Two Hands (6 to 10)	Present 5 on left hand + 1 on right (6) up to 5 on left + 5 on right (10).	Seamless dual-hand tracking and correct summed count.
Rotational Invariance	Rotate hand laterally 45° and 90° relative to camera axis.	Accurate counts maintained due to wrist distance ratios.
Camera Switching	Select different cameras from device selector dropdown.	Active stream stopped, new hardware initialized cleanly.
Audio Toggle	Mute/unmute sound feedback button.	AudioContext suspended/resumed without thread interruption.

12. Challenges Faced

- **Thumb Flexion vs. Abduction:** The thumb does not articulate vertically toward the wrist. Simple Y-axis checks misclassified curled thumbs as extended. *Solution:* Implemented comparative Euclidean distance ratios relative to Pinky MCP (17) and Index MCP (5).
- **Webcam Mirroring Alignment:** Unmirrored camera feeds confuse human users during spatial alignment. *Solution:* Applied horizontal canvas mirroring (scale(-1, 1)) while dynamically inverting MediaPipe handedness labels (Left vs. Right) to maintain intuitive user experience.
- **Boundary Flickering:** Fingers held halfway extended produced rapid count toggling. *Solution:* Developed an in-memory 5-frame rolling majority voting filter that smoothed transitions without perceivable latency.

13. Learnings & Key Takeaways

- **WebAssembly Transforms Edge AI:** Neural networks that previously required dedicated desktop installations can now run directly inside sandboxed browsers at full frame rates.
- **Heuristics + Deep Learning Synergy:** Combining neural regression (finding 21 landmarks) with deterministic vector geometry (distance checks) provides an optimal balance between accuracy and computational speed.
- **Zero-Framework Efficiency:** Vanilla HTML, CSS, and JavaScript provide clean codebases with zero dependency vulnerabilities and instant deployment.

14. Future Enhancements

- **Touch-Free Gesture Calculator:** Allow users to present two numbers separated by an operator gesture (e.g., peace sign for addition, thumbs up for equals) to perform touch-free arithmetic.
- **Continuous Dynamic Gestures:** Implement motion trajectory analysis to detect air-writing, swiping, and pinch-to-zoom gestures.
- **Multilingual Voice Output:** Integrate the browser Web Speech API to synthesize spoken number pronunciation in multiple languages.

15. References

1. Lugaresi, C., et al. 'MediaPipe: A Framework for Building Perception Pipelines.' *arXiv preprint arXiv:1906.08172* (2019).
2. Zhang, F., et al. 'MediaPipe Hands: On-device Real-time Hand Tracking.' *CVPR Workshop on Computer Vision for AR/VR* (2020).
3. W3C Media Capture and Streams Working Group. 'Media Capture and Streams (getUserMedia).' *W3C Recommendation* (2022).
4. W3C Audio Working Group. 'Web Audio API.' *W3C Recommendation* (2021).
5. Google Developers. 'MediaPipe Solutions - Hand Landmark Detection Guide.'
https://developers.google.com/mediapipe/solutions/vision/hand_landmarker